

MLOps on Databricks: A Comprehensive Guide

1. The Two Promotion Patterns

Core MLOps design decision on Databricks:

- **promote the trained model artifact or the code that trains it**
 - How the Feature Store is scoped
 - How serving endpoints are updated
 - Data governance
-

2. Deploy Code (Promote Code)

What It Is

In the **deploy code** pattern, the *training pipeline* — not the trained model artifact — is what moves through environments. The same reviewed, tested code runs independently in dev, staging, and production. Each environment trains its own model on its own data.

```
None
Git (dev branch) → PR → CI unit tests
                    → merge to main → staging integration tests + staging model
trained
                    → release branch → prod model trained on full prod data →
endpoint updated
```

When to Use It

Databricks **recommends deploy code in most situations**. Specifically use it when:

- Production data is accessible (directly or read-only) from a staging/prod environment
- Automated, scheduled model retraining is a requirement
- You need the production model guaranteed to be trained on production-grade data
- The team is building multi-component pipelines (featurization → training → inference → monitoring) that all need to be versioned together
- Regulatory or audit requirements demand that every production asset be traceable to a specific, reviewed code commit

Benefits

Benefit	Why It Matters
Full code review on everything in prod	No unreviewed model artifact ever enters production
Production model trained on prod data	Eliminates dev/prod training distribution skew
Automated retraining is safe by design	Retraining uses code that has already been approved and tested
Supporting pipelines (feature, inference, monitoring) stay in sync	All components promoted together via the same CI/CD pipeline
Single source of truth is Git	Every prod state can be reproduced from a commit hash

Limitations

Limitation	Mitigation
Steeper learning curve for data scientists	Use MLOps Stacks templates to reduce boilerplate; enforce via DAB targets
Training cost incurred in each environment	Limit staging to a data subset; use serverless compute to avoid idle spend
Data scientists must be able to view prod training results	Grant read-only access to prod MLflow experiments and inference tables
Reproducibility depends on data versioning	Use Delta Time Travel + <code>mlflow.log_input()</code> to pin the exact data version used
More complex CI/CD setup	MLOps Stacks scaffolds this for GitHub Actions and Azure DevOps out of the box

3. Deploy Artifact (Promote Model)

What It Is

In the **deploy artifact** pattern, training happens once in the development environment. The resulting model artifact is tested in staging (against a held-out data subset) and then — if it passes — promoted to production. Promotion is done via `MlflowClient.copy_model_version()` between Unity Catalog catalogs/workspaces.

None

```
Dev training → model logged to dev UC registry
               → staging: integration test against staging data
               → copy_model_version() → prod UC registry
               → prod endpoint alias updated → live
```

When to Use It

Use deploy artifact when one or more of the following apply:

- Model training is **prohibitively expensive or slow** (e.g., large foundation model fine-tuning, weeks-long GPU training runs)
- The model only needs to be trained **once** or very infrequently, and the serving/inference code changes more often than the model itself
- All work is done in a **single Databricks workspace** (dev and prod are catalog-separated, not workspace-separated)
- Regulatory requirements demand **model artifact immutability** — the exact binary that passed testing must be the exact binary deployed

Benefits

Benefit	Why It Matters
Training cost paid only once	Critical for expensive LLM fine-tuning or lengthy GPU jobs
Deterministic artifact	Exactly the model tested in staging is the model deployed to prod
Simpler for infrequent training cadences	When models train monthly or quarterly, full deploy-code overhead may not be warranted

Benefit	Why It Matters
Serving/monitoring pipelines can evolve independently	Code lifecycle decoupled from model lifecycle

Limitations

Limitation	Mitigation
Model not trained on production data	Grant dev/staging read access to prod catalog for training data
Automated retraining is harder to govern	The prod environment can't run the retraining without code review; risks unapproved model versions
Supporting pipeline code must be deployed separately	Requires additional CI/CD steps for feature, inference, and monitoring pipelines
<code>copy_model_version()</code> requires cross-workspace UC access setup	Set up workspace-catalog binding + service principal grants (see access setup below)
Lineage gap: model lineage shows dev data, not prod data	Partially mitigated by tagging model version with data snapshot reference
Not viable if prod data is inaccessible from dev	If prod data can't be read from dev for training, you must use deploy code

Cross-Workspace `copy_model_version` Access Setup

When using separate dev and prod workspaces, the prod service principal needs:

SQL

```
-- Metastore admin: bind dev catalog to prod workspace (read-only)
ALTER CATALOG dev_catalog SET BINDING READ_ONLY TO workspace <prod_workspace_id>;

-- Grant prod SP read access to source model
GRANT USE CATALOG ON CATALOG dev_catalog TO `prod-deploy-sp`;
GRANT USE SCHEMA ON SCHEMA dev_catalog.ml_models TO `prod-deploy-sp`;
GRANT EXECUTE ON MODEL dev_catalog.ml_models.my_model TO `prod-deploy-sp`;

-- Grant prod SP write access to destination
GRANT USE CATALOG ON CATALOG prod_catalog TO `prod-deploy-sp`;
GRANT USE SCHEMA ON SCHEMA prod_catalog.ml_models TO `prod-deploy-sp`;
GRANT CREATE MODEL ON SCHEMA prod_catalog.ml_models TO `prod-deploy-sp`;
```

Always use a **service principal**, never a human PAT token, for automated model promotion. Human PATs carry full user permissions, are not auditable as pipeline actions, and break when the user leaves.

4. Pattern Comparison at a Glance

Dimension	Deploy Code	Deploy Artifact
What is promoted	Training pipeline code	Trained model binary
Databricks recommendation	Default / most situations	Specific cases only
Training environment	Each env trains independently	Dev trains once
Prod model trained on prod data	Yes	Only if prod data accessible from dev
Automated retraining safety	High (code reviewed before retraining)	Lower (artifact reuse; retrain requires new copy cycle)
Training cost	Paid per environment	Paid once
CI/CD complexity	Higher	Lower (but additional artifact transfer step)
Primary promotion mechanism	DAB <code>databricks bundle deploy</code>	<code>MlflowClient.copy_model_version()</code>
Rollback mechanism	Git revert + redeploy	Reassign UC model alias to previous version
Audit trail	Git commit → pipeline run → model version	Model version copy lineage in UC

5. How Patterns Relate to the Feature Store

Architecture

Databricks **Feature Engineering in Unity Catalog** is the current offering. The legacy Workspace Feature Store is deprecated and unavailable for new workspaces. Any Delta table with a primary key in Unity Catalog can act as a feature table.

The Feature Store has two serving modes:

- **Offline Store:** Delta tables (historical, point-in-time correct) — used for model training and batch inference
- **Online Store:** Now backed by **Lakebase Autoscaling** (key-value store for sub-millisecond lookup) — used for real-time inference. Requires Databricks Runtime 16.4 LTS ML or above.

In Deploy Code

Feature tables are **environment-scoped**. Dev, staging, and prod each have their own catalog with their own feature tables:

None

```
dev_catalog.features.customer_features      (dev compute writes here)
staging_catalog.features.customer_features  (staging compute writes here)
prod_catalog.features.customer_features     (prod compute writes here)
```

The same featurization code runs in each environment, producing environment-appropriate feature tables. This ensures true isolation: staging tests against staging features, prod trains and serves from prod features. Online tables are published separately per environment.

Python

```
# Same code, different catalog resolved by DAB target variable
mlflow.set_registry_uri("databricks-uc")
feature_table = f"{catalog}.features.customer_features"
training_set = fe.create_training_set(
    df=labels_df,
    feature_lookups=[FeatureLookup(table_name=feature_table,
    lookup_key="customer_id")],
    label="churn"
)
```

In Deploy Artifact

Feature management is more complex — and the **online feature store introduces a critical additional dimension** that must be explicitly designed for.

Offline Feature Store

The model artifact is trained on dev feature data (offline Delta tables), but it must serve prod features at inference time. This creates risk if feature definitions or distributions differ between dev and prod.

Two mitigations:

1. **Grant dev read access to prod feature tables for training** — the model then trains on prod data while still running in dev compute. The embedded feature lookup metadata will point to `prod_catalog`, which is correct at serving time.
2. Use `mlflow.log_input()` to record the exact feature table version used in training, enabling reproducibility validation before promotion.

Online Feature Store: The Critical Gap

When you log a model via `fe.log_model()`, the **feature table names are embedded directly in the model artifact** as lookup metadata (e.g., `dev_catalog.features.customer_features`). When this artifact is copied to prod and served by Mosaic AI Model Serving, the serving infrastructure uses those embedded references to fetch features from the online store at inference time.

This creates a **hardcoded catalog reference problem**:

```
None
Model trained in dev → feature lookup embedded:
dev_catalog.features.customer_features
    ↓ copy_model_version()
Model served in prod → still resolves: dev_catalog.features.customer_features
(WRONG)
                        should resolve: prod_catalog.features.customer_features
```

If the dev online store and prod online store are separate (which they should be for isolation), serving the promoted artifact from prod will hit the **dev online store** — stale, lower SLA, potentially missing recent data — unless the lookup is explicitly pointed at prod.

Solutions for Online Feature Store in Deploy Artifact

Option: Train against prod feature tables from dev compute

Grant the dev workspace read-only access to the prod catalog feature tables and online store. The model artifact then embeds `prod_catalog` references and resolves correctly after promotion — no discrepancy between training and serving.

SQL

```
-- Bind prod catalog to dev workspace (read-only)
ALTER CATALOG prod_catalog SET BINDING READ_ONLY TO workspace <dev_workspace_id>;

-- Grant dev training SP access to prod feature tables
GRANT USE CATALOG ON CATALOG prod_catalog TO `dev-training-sp`;
GRANT USE SCHEMA ON SCHEMA prod_catalog.features TO `dev-training-sp`;
GRANT SELECT ON TABLE prod_catalog.features.customer_features TO
`dev-training-sp`;
```

Python

```
# Train in dev but point to prod feature table - artifact embeds prod_catalog
reference
feature_table = "prod_catalog.features.customer_features"
training_set = fe.create_training_set(
    df=labels_df,
    feature_lookups=[FeatureLookup(table_name=feature_table,
    lookup_key="customer_id")],
    label="churn"
)
fe.log_model(
    model=model,
    artifact_path="model",
    flavor=mlflow.sklearn,
    training_set=training_set,
    registered_model_name="dev_catalog.ml_models.churn_model"
)
# After copy_model_version() to prod, the serving endpoint resolves prod_catalog
online store ✓
```

Option 2: Use FeatureSpec with on-demand Unity Catalog functions

Instead of embedding offline table lookups, define features as **Unity Catalog SQL functions** (`FeatureSpec`). The function is resolved at inference time, not baked into the artifact. This decouples the model artifact from catalog-specific table references entirely.

Python

```
from databricks.feature_engineering import FeatureFunction, FeatureLookup
```



```

# UC function handles the lookup logic; catalog is resolved at runtime by the
serving endpoint's context
feature_spec = FeatureSpec(
    features=[
        FeatureFunction(
            udf_name="prod_catalog.features.compute_customer_features",
            input_bindings={"customer_id": "customer_id"}
        )
    ]
)

```

This is the most flexible approach — the function itself can be scoped per environment, and the model artifact remains catalog-agnostic.

Online Feature Store Environment Comparison

Aspect	Dev Online Store	Prod Online Store
Backing	Lakebase Autoscaling (DBR 16.4 LTS ML+)	Lakebase Autoscaling (DBR 16.4 LTS ML+)
Refresh cadence	Lower frequency, smaller scale	High frequency; SLA-bound freshness
Data volume	Subset or sample	Full entity population
Access	Dev service principal + data scientists	Prod service principal only; strict SP-only writes
Cost	Smaller instance, scale-to-zero acceptable	Sized for production traffic; SLA requires minimum replicas
Who writes	Dev featurization pipeline	Prod featurization pipeline (CI/CD managed)

The Feature Store's MLflow integration automatically records feature dependencies in the model artifact. At inference, the serving endpoint resolves features from the online store using those embedded references.

Python

```
# After promotion: prod serving endpoint picks up the copied model version
# and resolves features from prod_catalog online store (if Option 1 was used at
# training time)
fe.log_model(
    model=model,
    artifact_path="model",
    flavor=mlflow.sklearn,
    training_set=training_set,
    registered_model_name="dev_catalog.ml_models.churn_model"
)
# At inference, serving endpoint resolves: prod_catalog.features.customer_features
# → prod online store ✓
```

Feature Store Governance in Both Patterns

- Feature tables in UC inherit full UC access controls (GRANT/REVOKE, row filters, column masks)
- Feature lineage (which models use which features, which source tables feed which features) is tracked automatically in UC Lineage graph
- Online tables (Lakebase-backed) are governed separately — publish only the features the serving endpoint needs, with scoped SP access

6. How Patterns Relate to Model Serving

Mosaic AI Model Serving

The current serving stack. Serverless, auto-scales to zero, integrates natively with UC Model Registry. Models are referenced by **alias** (e.g., **Champion**, **Challenger**) rather than version number, decoupling serving configuration from deployment mechanics.

Python

```
# Endpoint always follows the "Champion" alias - no endpoint reconfiguration
# needed on model update
client.set_registered_model_alias("prod_catalog.ml_models.churn_model",
    "Champion", new_version)
```

Inference Tables automatically log inputs and outputs (adds ~10% cost overhead but essential for monitoring and compliance).

In Deploy Code

The CI/CD pipeline (running as a service principal) handles the full promotion:

1. PR merged → staging DAB deploy → staging training job runs → staging model registered to `staging_catalog`
2. Integration tests pass → release branch → prod DAB deploy → prod training job runs → prod model registered to `prod_catalog`
3. Post-training job: alias `Champion` reassigned to new version → serving endpoint automatically picks up new version

The serving endpoint definition itself lives in the DAB configuration (`databricks.yml`), versioned in Git.

```
None
# databricks.yml (DAB resource definition)
resources:
  model_serving_endpoints:
    churn_model_endpoint:
      name: churn-model-prod
      config:
        served_entities:
          - entity_name: prod_catalog.ml_models.churn_model
            entity_version: ${var.champion_version}
            workload_size: Small
            scale_to_zero_enabled: true
```

In Deploy Artifact

After `copy_model_version()` transfers the artifact to the prod catalog, a separate step reassigns the alias and optionally triggers a canary deployment:

```
Python
# CI/CD step after successful copy
client.set_registered_model_alias(
    "prod_catalog.ml_models.churn_model", "Champion", copied_version.version
)

# Gradual rollout: route 10% to new Challenger before full promotion
# Use traffic_config in the serving endpoint to split Champion/Challenger traffic
```

A/B testing and blue/green deployments are supported in both patterns via `traffic_config` on the serving endpoint. The pattern does not change the serving mechanics — it only changes how the registered model version was produced.

7. How Patterns Relate to Data Governance and Lineage

Unity Catalog as the Governance Backbone

UC governs all MLOps assets — data tables, feature tables, model versions, experiments, notebooks, volumes, and serving endpoints — under a single permission model.

- Models are registered as `FUNCTION` securables in UC → governed via `GRANT EXECUTE, GRANT MANAGE`
- `USE CATALOG + USE SCHEMA` are always prerequisites for any model access
- Workspace-catalog binding controls which workspaces can see which catalogs, irrespective of user-level grants

Lineage

Automatic lineage is captured when:

1. Training data is logged via `mlflow.log_input()`:

```
Python
dataset = mlflow.data.load_delta(table_name="prod_catalog.ml_data.train",
version="5")
with mlflow.start_run():
    mlflow.log_input(dataset, "training")
    mlflow.sklearn.log_model(...,
registered_model_name="prod_catalog.ml_models.churn_model")
```

→ UC Lineage graph shows: `train_table (v5)` → `MLflow Run` → `churn_model (v3)`

2. Feature Store APIs are used (lineage captured automatically, no extra code needed): →
UC Lineage graph shows: `raw_events_table` → `customer_features` →
`churn_model (v3)` → `churn_model_endpoint`

The Lineage tab on the model version page in Catalog Explorer surfaces the full upstream graph visually.

Lineage Comparison by Pattern

Lineage Node	Deploy Code	Deploy Artifact
Training data → model	Prod data version → prod model	Dev data version → dev model
Feature table → model	Prod feature table version	Dev feature table version (gap if model promoted to prod)
Model → serving endpoint	Direct (same catalog)	Via copy; <code>source_model_version</code> tag on copied version
Audit: who registered/promoted	Service principal (CI/CD run ID)	Service principal (CI/CD run ID) if automated; human if not

Governance Best Practices (Both Patterns)

- Run all automated promotion jobs as **named service principals**, never human users. UC audit logs attribute every action to an identity — SP names should reflect the pipeline, e.g., `mlops-prod-deploy-sp`.
- Use **model version tags** for approval gates: `validation_status=approved`, `promoted_by=ci-pipeline-run-123`
- Use **aliases** (`Champion`, `Challenger`, `Baseline`) rather than version numbers in serving and batch inference — they decouple deployment from versioning and simplify rollbacks
- Enable **Inference Tables** on serving endpoints for post-deployment lineage: inference inputs/outputs are written back to Delta tables in UC, enabling drift detection via Lakehouse Monitoring
- Scope the prod catalog with **strict MODIFY restrictions** — only service principals should write to prod; data scientists should have read-only access for analysis

8. Supported Tech Stack (2025–2026)

The following table reflects the current Databricks MLOps Stacks reference architecture:

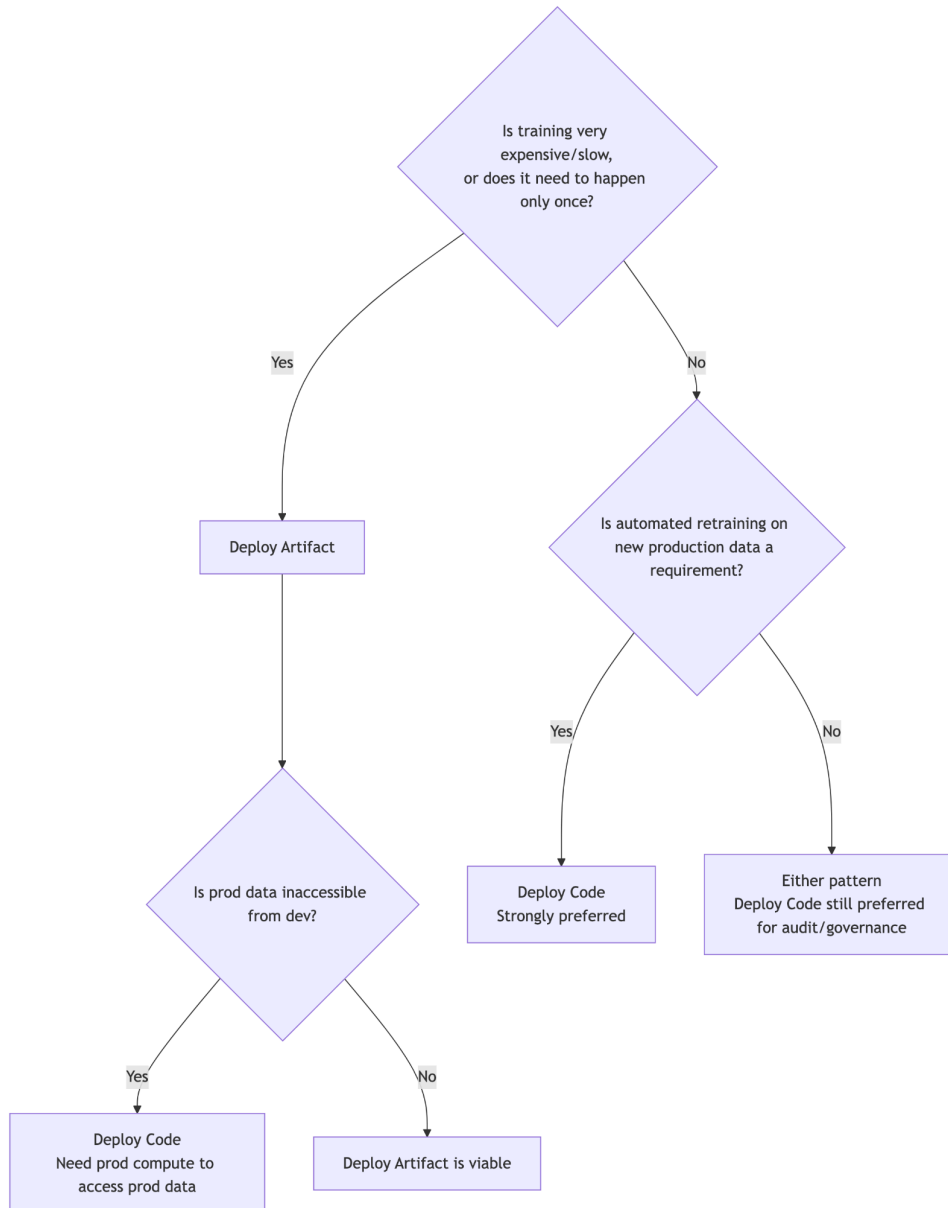
MLOps Component	Databricks Tool	Notes
Experiment tracking	MLflow Tracking	Managed; <code>mlflow.set_registry_uri("databricks-uc")</code> for UC registry
Model registry	Models in Unity Catalog	Replaces Workspace Model Registry; FUNCTION securable
Model lifecycle (stages)	UC Aliases (<code>Champion</code> , <code>Challenger</code>)	Replaces MLflow <code>stage=Production/Staging</code> ; stages deprecated in UC
Feature management	Feature Engineering in Unity Catalog	Any Delta table with PK; legacy Workspace Feature Store deprecated
Online feature serving	Online Feature Stores (Lakebase Autoscaling)	Requires DBR 16.4 LTS ML+; low-latency KV store
Real-time model serving	Mosaic AI Model Serving	Serverless; auto-scales to zero; alias-based routing
Batch inference	Lakeflow Jobs	Scheduled notebooks or Python scripts; Feature Store <code>score_batch()</code>
Infrastructure as code	Databricks Asset Bundles (DABs)	<code>databricks.yml</code> ; versioned in Git; replaces legacy Stacks CLI
Project scaffolding	MLOps Stacks (<code>databricks bundle init mlops-stacks</code>)	Generates DAB project with CI/CD pre-configured
Orchestration	Lakeflow Jobs (formerly Databricks Workflows)	Supports tasks, dependencies, triggers
CI/CD	GitHub Actions or Azure DevOps	DAB deploy commands in workflow YAML
Governance	Unity Catalog	Data, models, features, notebooks — all governed

MLOps Component	Databricks Tool	Notes
Monitoring	Lakehouse Monitoring	Drift detection; data quality; model performance over inference tables
Authentication (automation)	Service Principals + OAuth M2M	Never PAT tokens in CI/CD; use Databricks Secrets for SP credentials
IaC for workspaces	Terraform (Databricks Provider)	Workspace provisioning, UC setup, SP creation
Cross-region/account sharing	Delta Sharing (D2D)	For model/feature sharing across metastores
SDK	Databricks SDK (<code>databricks-sdk</code>)	Python, Java, Go; programmatic access to all APIs

Key Version Requirements

Component	Minimum Version
MLflow (for UC model registry)	<code>>=3.4.0</code> (for <code>copy_model_version</code> with UC)
Databricks CLI (for DABs)	<code>>=0.212.2</code>
Databricks Runtime ML (Online Feature Store)	<code>>=16.4 LTS ML</code>
Databricks Runtime ML (Feature Engineering in UC)	<code>>=13.3 ML</code>

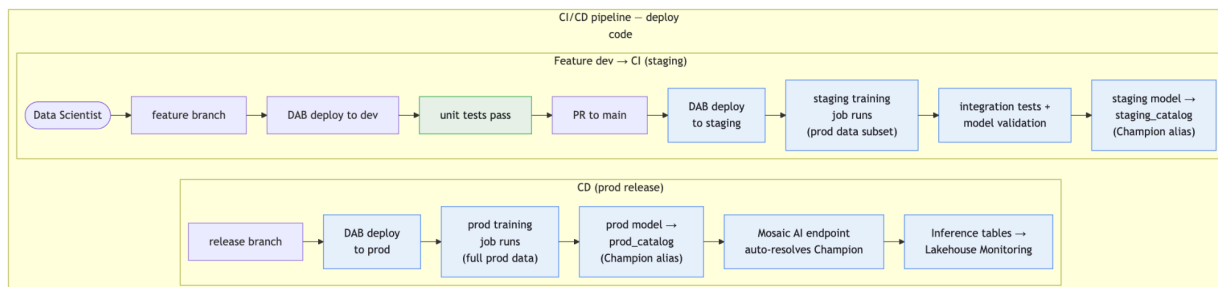
9. Choosing Your Pattern: Decision Tree



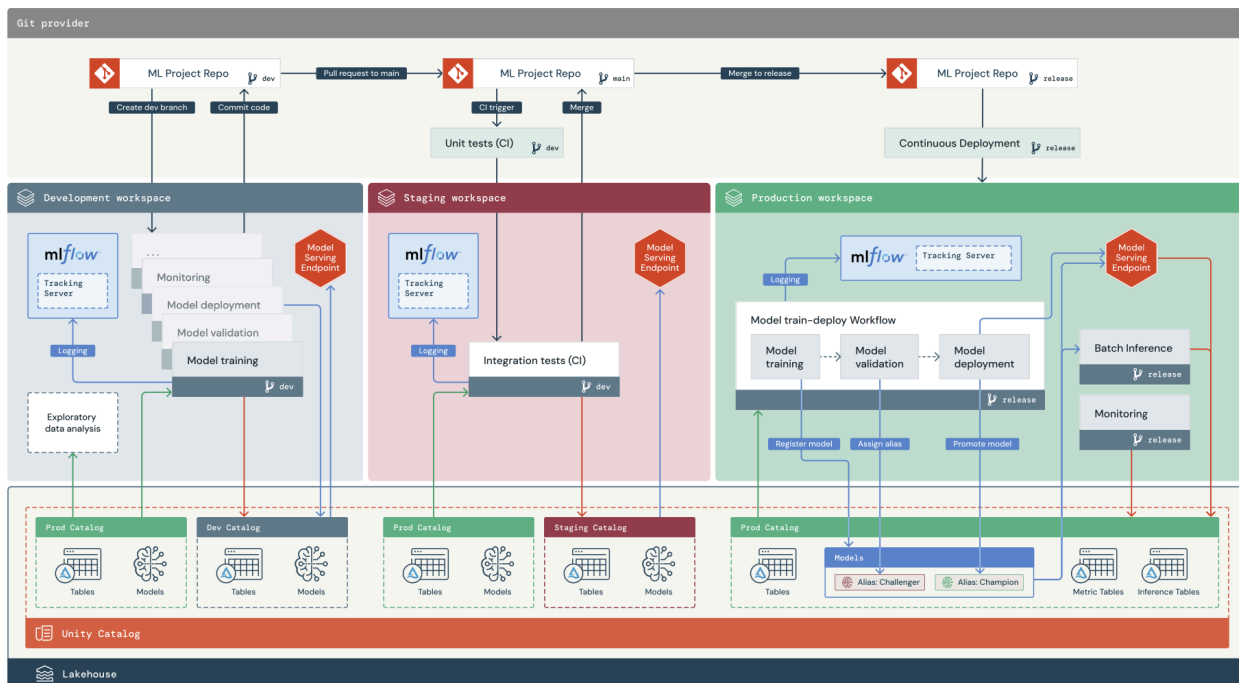
For most enterprise ML teams building on Databricks in 2025–2026, **deploy code + DABs + Feature Engineering in UC + Mosaic AI Model Serving** is the reference stack. Deploy artifact remains the right choice for LLM/foundation model fine-tuning workflows where retraining costs are prohibitive.

10. End-to-End Flow Summary

Deploy Code (Reference Architecture)

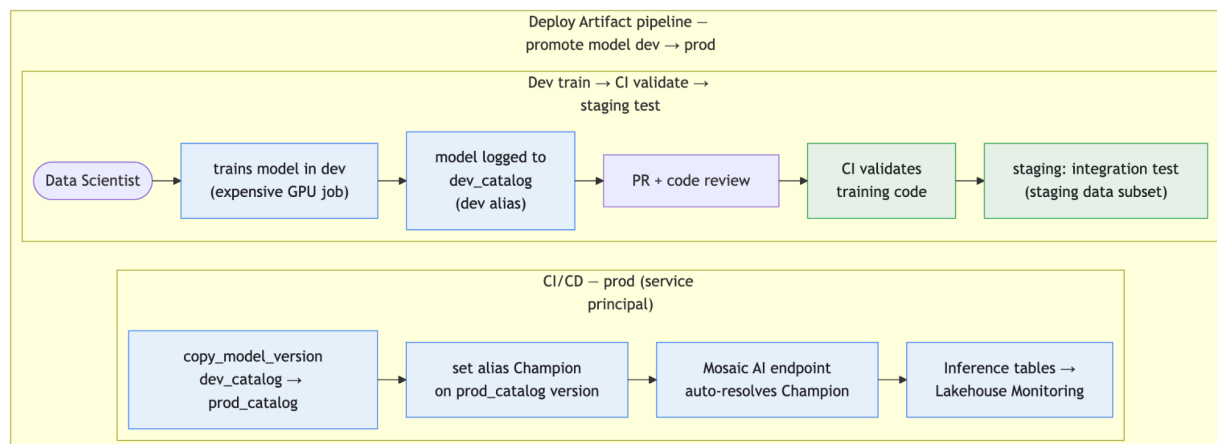


Flow Diagram



MLOps Stack Reference Architecture

Deploy Artifact (Specific Cases)



Flow Diagram

11. MLflow Export/Import: Filling Gaps in the Deploy Artifact Pattern

What It Is

[mlflow-export-import](#) is an open-source community tool (hosted under the [mlflow](#) GitHub org) that provides fine-grained export, import, and copy operations for MLflow objects — runs, experiments, registered models, and model versions — across Databricks workspaces, Unity Catalog metastores, and even external MLflow tracking servers (e.g., Amazon SageMaker MLflow).

This is a field/community solution, not an officially supported Databricks product. It is not covered by Databricks SLAs. Evaluate its use carefully for production pipelines; pair it with a service principal and audit logging.

Gaps It Fills vs. Native `copy_model_version()`

The native `MlflowClient.copy_model_version()` (MLflow $\geq 3.4.0$) has important limitations that `mlflow-export-import` addresses:

Capability	Native <code>copy_model_version()</code>	<code>mlflow-export-import</code>
Copy model artifact	Yes	Yes
Copy the associated MLflow run	No (shallow only)	Yes (deep copy option)
Cross-workspace copy (same metastore)	Yes	Yes
Cross-workspace, cross-metastore copy	No	Yes
Cross-account / cross-region copy	No	Yes (via shared cloud bucket)
WS Model Registry $\rightarrow$ UC Model Registry	Yes (limited)	Yes (explicit support)
Bulk export/import (all models or experiments)	No	Yes
Lineage tag preservation	No	Yes (<code>--copy-lineage-tags</code>)
Disaster recovery / backup	No	Yes

The most impactful gap: native `copy_model_version()` performs a shallow copy — the destination model version points to the source run, which lives in the source workspace. If the

source workspace is decommissioned or the run is deleted, lineage breaks.

`mlflow-export-import`'s deep copy creates a full replica of the run in the destination workspace, giving the promoted version an independent, self-contained lineage record.

Key Use Cases

1. Cross-metastore promotion (separate Databricks accounts or regions)

When dev and prod live in different Databricks accounts (and therefore different UC metastores), native `copy_model_version()` cannot bridge the boundary.

`mlflow-export-import` uses an intermediate shared cloud bucket (S3, ADLS, or GCS) as a staging area:

Shell

```
# Deep copy: UC model version from dev metastore to prod metastore
copy-model-version \
  --src-model dev_catalog.ml_models.churn_model \
  --src-version 12 \
  --dst-model prod_catalog.ml_models.churn_model \
  --dst-experiment-name /prod-mlops/experiments/churn \
  --src-registry-uri databricks-uc://dev-profile \
  --dst-registry-uri databricks-uc://prod-profile \
  --copy-lineage-tags true
```

2. Preserving run lineage on promotion (deep copy)

With `--dst-experiment-name` specified, the tool creates a full copy of the source run — metrics, params, tags, artifacts — in the destination workspace. The promoted model version points to this replicated run, not the original dev run.

Shell

```
# Shallow copy (default, same as native API) - run stays in source workspace
copy-model-version --src-model ... --dst-model ... --src-registry-uri ...
--dst-registry-uri ...

# Deep copy - run is replicated to destination workspace (recommended for
audit/lineage)
copy-model-version --src-model ... --dst-model ... \
  --dst-experiment-name /prod-mlops/experiments/churn \   # triggers run
  replication
  --copy-lineage-tags true                                # stamps source
  provenance on destination version
```

The `--copy-lineage-tags` flag adds `mlflow_exim.*` tags to the destination version,

stamping source provenance:

```
None
mlflow_exim.src_version.name:    dev_catalog.ml_models.churn_model
mlflow_exim.src_version.version: 12
mlflow_exim.src_version.run_id:  c62ccf932e0649a2b9247cc76d89b637
mlflow_exim.src_run.workspaceURL: dev-workspace.azuredatabricks.net
```

3. Workspace Model Registry → Unity Catalog migration

For teams still on the legacy Workspace Model Registry migrating to UC:

```
Shell
copy-model-version \
  --src-model OldWorkspaceModel \
  --src-version 5 \
  --dst-model prod_catalog.ml_models.churn_model \
  --src-registry-uri databricks://legacy-workspace-profile \
  --dst-registry-uri databricks-uc://prod-profile
```

Note: source model version must have a model signature to satisfy UC requirements. Use `MLFLOW_SKIP_SIGNATURE_CHECK_FOR_UC_REGISTRY_MIGRATION=true` only for migration scenarios.

4. Bulk export for disaster recovery / backup

```
Shell
# Export all registered models from a workspace to a shared cloud bucket
export-models --models "model_a,model_b,model_c" --output-dir
s3://mlops-backup/models/

# Import into a recovery or new workspace
import-models --input-dir s3://mlops-backup/models/ \
  --delete-model true \
  --dst-registry-uri databricks-uc://recovery-profile
```

Installation

```
Shell
# Install directly from GitHub (no PyPI release; use the mlflow org repo)
pip install
git+https://github.com/mlflow/mlflow-export-import/#egg=mlflow-export-import
```

For use within a Databricks notebook or job, install as a cluster library or via `%pip install`.

When to Use It vs. Native `copy_model_version()`

Scenario	Use
Same metastore, cross-workspace (standard deploy artifact)	Native <code>copy_model_version()</code> — simpler, supported
Need full run lineage in destination workspace	<code>mlflow-export-import</code> deep copy
Cross-metastore or cross-account promotion	<code>mlflow-export-import</code> — native API cannot do this
WS Model Registry → UC migration	Either; <code>mlflow-export-import</code> handles bulk and lineage better
Disaster recovery / backup of all models	<code>mlflow-export-import</code> bulk tools
Automated CI/CD in production	Native <code>copy_model_version()</code> preferred (supported); use <code>mlflow-export-import</code> for edge cases only

Operational Considerations

- Requires a shared cloud storage bucket accessible from both source and destination workspaces for cross-workspace deep copies
 - Authenticate using `~/databrickscfg` profiles or environment variables (`DATABRICKS_HOST`, `DATABRICKS_TOKEN`) — in CI/CD, use a service principal token stored in Databricks Secrets
 - Lineage tags are only maintained for one hop — if you copy a version that was itself already a copy, previous lineage tags are overwritten
 - Not suitable as a replacement for a full CI/CD deploy-code pipeline; best used for targeted migration, recovery, or edge-case promotion scenarios
-

12. References

Architecture & MLOps Strategy

- [MLOps Workflows on Databricks](#) — recommended end-to-end workflow including environment separation and CI/CD integration
- [Model Deployment Patterns](#) — official comparison of deploy code vs. deploy artifact with decision criteria
- [The Big Book of MLOps](#) — comprehensive ebook covering MLOps architecture, patterns, and trade-offs on Databricks

Model Lifecycle & Registry

- [Manage Model Lifecycle in Unity Catalog](#) — model registration, aliases, lineage tracking, and cross-workspace access
- [Migrate Model Versions from Workspace Registry to Unity Catalog](#) — `copy_model_version()` usage and migration guidance
- [Models in Unity Catalog — Privileges and Access Control](#) — GRANT/REVOKE for FUNCTION securables (models)

Feature Store

- [Feature Engineering in Unity Catalog](#) — current feature store offering; replaces legacy Workspace Feature Store
- [Databricks Online Feature Stores](#) — Lakebase Autoscaling-backed online store for real-time inference (DBR 16.4 LTS ML+)
- [Feature Governance and Lineage](#) — automatic lineage capture via Feature Store APIs and UC

Model Serving & Monitoring

- [Mosaic AI Model Serving](#) — serverless real-time serving, alias-based routing, inference tables
- [Lakehouse Monitoring](#) — drift detection, data quality, and model performance monitoring over inference tables

Infrastructure as Code & CI/CD

- [What are Databricks Asset Bundles \(DABs\)?](#) — IaC for ML resources; `databricks.yml` configuration reference

- [MLOps Stacks — Asset Bundles](#) — scaffolding a production-grade MLOps project with CI/CD pre-configured
- [MLOps Stacks GitHub Repository](#) — open-source templates for training, batch inference, and CI/CD workflows
- [Best Practices and Recommended CI/CD Workflows on Databricks](#) — CI/CD patterns for multi-team ML collaboration using DABs

Community & Field Tools

- [mlflow-export-import \(GitHub\)](#) — open-source tool for cross-workspace/cross-metastore MLflow object migration; deep copy, bulk export, and lineage tag support

Governance & Unity Catalog

- [Unity Catalog Overview](#) — centralized governance for data, models, features, and AI assets
- [Workspace-Catalog Binding](#) — restricting catalog access to specific workspaces for environment isolation
- [Unity Catalog Privileges Reference](#) — full privilege matrix including USE CATALOG, USE SCHEMA, EXECUTE, CREATE MODEL